```
            source_ip = match.group("ip").strip()
            hex_data = match.group("hex").strip()
            ascii_data = match.group("ascii").strip().upper()

            try:
                payload = bytes.fromhex(hex_data)
            except ValueError:
                continue

            tokens = [t.strip() for t in ascii_data.split(",") if t.strip()]
            callsign_hint = tokens[0] if len(tokens) == 1 else None
            rows.append((raw_time, source_ip, payload, callsign_hint))

        # Backward compatibility for logs that do not include ASCII section.
        if not rows:
            for match in full_log_hex_only_pattern.finditer(text):
                raw_time = match.group("time").strip()
                source_ip = match.group("ip").strip()
                hex_data = match.group("hex").strip()

                try:
                    payload = bytes.fromhex(hex_data)
                except ValueError:
                    continue

                rows.append((raw_time, source_ip, payload, None))

        if not rows:
            base_time = datetime.utcnow()
            for idx, match in enumerate(simple_pattern.finditer(text)):
                hex_data = match.group("hex").strip()
                ascii_data = match.group("ascii").strip().upper()

                try:
                    payload = bytes.fromhex(hex_data)
                except ValueError:
                    continue

                # ASCII line may contain multiple tokens; first token is the callsign candidate.
                tokens = [t.strip() for t in ascii_data.split(",") if t.strip()]
                callsign_hint = tokens[0] if tokens else None
                timestamp = (base_time + timedelta(seconds=idx)).isoformat() + "Z"
                rows.append((timestamp, "", payload, callsign_hint))

        # New compact format:
        # <hex_line>
        #
        # <ascii_line>
        if not rows:
            hex_line_re = re.compile(r"^[0-9a-fA-F]{10,}$")
            clean_lines = []
            for line in text.splitlines():
                stripped = line.strip()
                if not stripped:
                    continue
                if stripped.startswith("="):
                    continue
                if stripped.upper() == "PINGSTATION 3 LIVE DATA LOG":
                    continue
                clean_lines.append(stripped)

            base_time = datetime.utcnow()
            idx = 0
            event_idx = 0
            while idx < len(clean_lines):
                hex_candidate = clean_lines[idx]
                if not hex_line_re.match(hex_candidate):
                    idx += 1
                    continue

                try:
                    payload = bytes.fromhex(hex_candidate)
                except ValueError:
```